Web Engineering & Development (SWE 363)

Back-End Development Fundamentals

Announcements

- Midterm on Thursday, **April 2nd**
  - 7:00-9:00 PM
  - Location: B22 (Rooms 119, 125, 127, 130, 134)

In today's lecture:

- Full-stack development (Node)

Reference:

- Zybook: 6.1

6.1 Full-stack Development

What is Full-Stack Development?

Front-End (Client-Side)

- Code that runs **in the browser**
- What users **see and interact with**
- Built with: **React**, HTML, CSS, JavaScript
- Examples: buttons, forms, navigation

Back-End (Server-Side)

- Code that runs **on the server**
- Handles **data and business logic**
- Built with: **Node.js + Express**
- Examples: databases, APIs, authentication

Front-End vs Back-End

Front-End

User interface

Running in browser

React components

Back-End

Server logic

Running on server

Handles requests

Full-Stack

Both together!

Complete applications

What is Node.js?

> **Node.js** is a JavaScript runtime environment

- Allows you to run JavaScript **outside the browser**
- Makes it possible to write **back-end code** using JavaScript
- Same language for front-end and back-end!

Before Node.js

Without Node.js:

Front-End:

- JavaScript

Back-End:

- Python, Java, PHP, etc.

Different languages!

Now with Node.js:

Front-End:

- JavaScript

Back-End:

- JavaScript

Same language!

How Does Node.js Help Back-End?

Node.js allows the server to:

- **Handle server requests** from browsers
- **Read/write files** on the server
- **Respond with data** to the browser
- **Manage databases** and APIs

What is Express?

> **Express** is a minimal web framework built on top of Node.js

- Makes back-end development **easier**
- Provides tools to create **routes**
- Handles **requests** and **responses** simply

Why Use Express?

- **Simple** to create APIs
- **Handles JSON** easily
- **Less code** than raw Node.js
- **Fast** to set up routes

Request and Response

Request

When the front-end asks for something

Example:

> "Give me all students"

Browser sends:

```
GET /api/students
```

Response

When the server sends back the answer

Example:

> Server responds with JSON

```
[
  { "id": 1, "name": "Ali" },
  { "id": 2, "name": "Sara" }
]
```

HTTP Methods

Two main methods we'll use:

Method	Purpose	Example
GET	Request data	"Show me all students"
POST	Send data	"Add a new student"

Creating Routes in Express

A **route** is a path on the server that handles requests

```
// GET route – request data
app.get('/api/students', (req, res) => {
  res.json(students);
});
```

- `/api/students` → the URL path
- `req` → the request from front-end
- `res` → the response we send back

POST Route Example

```
// POST route - send data to server
app.post('/api/students', (req, res) => {
  const newStudent = req.body; // data from front-end
  // Add to database/array
  students.push(newStudent);
  res.json(newStudent); // send back the new student
});
```

How Data Flows

1. Front-End

```
fetch('http://localhost:3000/api/students')
  .then(r => r.json())
  .then(data => setStudents(data));
```

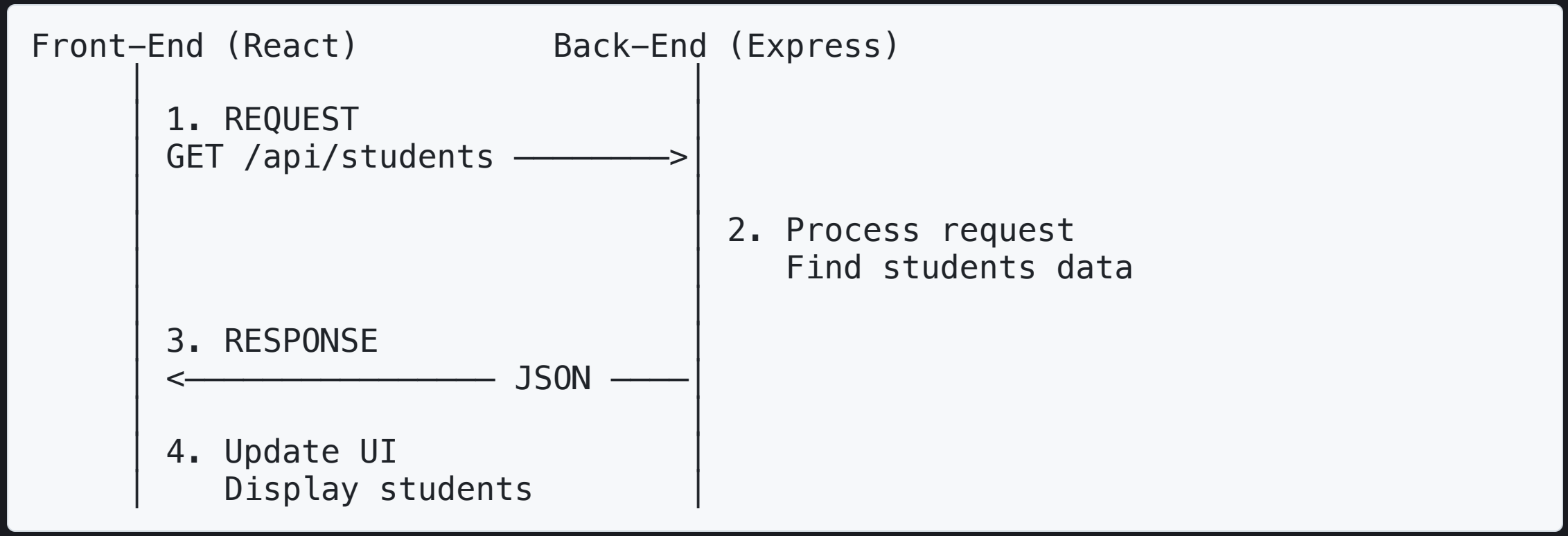
Sends REQUEST →

2. Back-End

```
app.get('/api/students', (req, res) => {
  res.json(students);
});
```

← Sends RESPONSE

Complete Flow Diagram



Testing the Back-End

1. **Start the server:**

```
node server.js
```

2. **Open browser:**

```
http://localhost:3000/api/students
```

3. **See JSON directly in browser!**

Key Takeaways

- **Full-stack** = Front-end + Back-end
- **Node.js** = Run JavaScript on server
- **Express** = Framework to build APIs easily
- **Request** = Front-end asks for data
- **Response** = Back-end sends data (JSON)
- **Routes** = Server paths that handle requests

30m

Demo

6.1 Introduction to Node

Next Class

- More on NodeJS